

Roteiro de Apresentação Técnica

Como defender o projeto em uma banca/entrevista

Material de estudo para apresentar e defender a solução. Traz o pitch inicial, as decisões de arquitetura e suas justificativas, o mapa que conecta diagramas a endpoints, um banco de perguntas prováveis com respostas, um roteiro de demonstração ao vivo e as limitações conhecidas. Estude com a pergunta-guia do roteiro em mente: “se este projeto fosse apresentado em uma entrevista técnica, ele se sustentaria?”.

1. Pitch (60–90 segundos)

“Construí o backend de uma rede de lanchonetes **multiunidade**. É uma API REST em FastAPI organizada em **DDD** (domínio, aplicação, infraestrutura e API). O fluxo crítico é **pedido** → **pagamento mock** → **atualização de status**, com **estoque por unidade**, baixa automática no pagamento aprovado, **fidelidade**, **autenticação JWT** com **autorização por perfil** e **trilha de auditoria**. Erros seguem um padrão único, há validação de contrato, paginação e 17 testes automatizados, além de coleção Postman e Swagger. Roda com um comando via Docker.”

2. Decisões de arquitetura e por quê

Decisão	Justificativa
DDD em 4 camadas	Separa regra de negócio de framework/persistência; testável e manutenível. O roteiro aceita DDD.
Entidades ricas (não anêmicas)	Invariantes ficam na entidade (ex.: máquina de estados do Pedido, estoque não-negativo).
Interfaces de repositório (ports)	Domínio não conhece SQLAlchemy; permite trocar persistência e testar com fakes.
Money em centavos (int)	Evita erros de arredondamento de ponto flutuante em dinheiro.
Estoque por unidade	A rede é multiunidade; cada loja tem estoque próprio (requisito do caso).
Baixa de estoque só no pagamento	Não reserva estoque de pedido que pode nunca ser pago; cancelamento repõe.
RBAC com require_roles	Atende MVP (auth + 1 regra de autorização) e gera 403 coerente.
Padrão de erro único	Contrato previsível para o front; inclui error, message, details, timestamp, path.
Pagamento mock determinístico	Campo 'aprovar' permite demonstrar sucesso e falha sem provedor real.
SQLite + create_all + seed	Zero configuração para correção; DATABASE_URL troca para Postgres.

3. Como os diagramas se conectam ao código

- **Casos de uso** (Cliente, Atendente, Cozinha, Gerente/Admin, Gateway) ■ viram **services** em application/ e rotas na API. Ex.: “Realizar Pedido” = PedidoService.criar + POST /pedidos.
- **DER** ■ modelos ORM em infrastructure/models.py (mesmas tabelas, PK/FK, UNIQUE de estoque por unidade e de pagamento 1:1).
- **Diagrama de classes (domínio)** ■ entidades em domain/entities + value objects. Métodos das classes = comportamentos (aplicar status, baixar estoque).
- **Sequência do fluxo crítico** ■ PagamentoService.processar: valida pedido, cria pagamento, marca PAGO, baixa estoque e acumula pontos.

4. Perguntas prováveis (e respostas)

Arquitetura

Por que DDD e não só MVC? O roteiro aceita ambos; escolhi DDD para isolar as regras de negócio em entidades testáveis e independentes de framework. A regra de dependência (tudo aponta para o domínio) facilita evolução e testes.

O que é um Value Object aqui? Money e Email: imutáveis, sem identidade, validam a si mesmos (Money em centavos; Email valida formato).

Regras de negócio

Como garante que não vende sem estoque? Na criação, o EstoqueService valida a disponibilidade na unidade antes de persistir; a baixa efetiva ocorre no pagamento aprovado (Estoqueltem.saida valida de novo a invariante).

E a idempotência do pagamento? Um pedido só pode ter um pagamento; uma segunda chamada retorna 409 PAGAMENTO_JA_PROCESSADO. (Melhoria: chave de idempotência por header para retries de rede.)

Transição de status? Há uma máquina de estados explícita; transições inválidas retornam 422. Cancelar repõe o estoque se o pedido já estava pago.

Concorrência, transações e escala

Dois pedidos simultâneos do último item? Hoje há janela de corrida entre validar e baixar. Em produção eu usaria bloqueio (SELECT ... FOR UPDATE) ou controle otimista por versão. Está documentado como melhoria.

O pagamento é atômico? Hoje cada repositório faz commit. O ideal é um **Unit of Work** envolvendo criar pagamento + mudar status + baixar estoque em uma única transação, com rollback em falha.

Desempenho em pico? Listagens são paginadas (page/limit); índices em chaves e e-mail. Escala horizontal exigiria banco gerenciado e cache.

Segurança e LGPD

Onde está a LGPD na prática? Coleta mínima (nome, e-mail), consentimento obrigatório, senha só em hash bcrypt, sem expor dados sensíveis nas respostas, autorização por perfil e auditoria de ações sensíveis. Direito ao esquecimento/anonimização ficou como evolução.

Por que o user_id vem do token? Evita que um usuário crie pedido em nome de outro; a identidade é sempre derivada do JWT, não do corpo.

Entrega/tecnologia

Por que SQLite? Reprodutibilidade imediata para correção; a DATABASE_URL troca para Postgres sem mudar o domínio. **Migrations?** Uso create_all + seed no escopo; em produção, Alembic.

5. Roteiro de demonstração ao vivo

```
1) docker compose up --build      # ou uvicorn app.main:app --reload
2) Abrir http://127.0.0.1:8000/docs (Swagger)
3) POST /auth/login (admin@raizes.com / admin123) -> copiar accessToken
4) GET /unidades e GET /unidades/1/cardapio
5) POST /auth/register (cliente, consentimentoLgpd=true) e login
```

```
6) POST /pedidos (canalPedido, unidadeId, itens) -> 201
7) POST /pagamentos/{pedidoId} {"aprovar": true} -> APROVADO
8) GET /estoque/1 (mostra a baixa) + GET /fidelidade/saldo
9) PATCH /pedidos/{id}/status (admin) EM_PREPARO->PRONTO->ENTREGUE
10) Mostrar ERROS: sem token (401), cliente cria produto (403),
    estoque insuficiente (409), canalPedido ausente (422)
11) GET /auditoria (admin) -> trilha das ações
12) Rodar pytest -q (17 testes verdes)
```

6. Limitações conhecidas (seja honesto na banca)

- Sem Unit of Work (commits separados no fluxo de pagamento).
- Sem controle de concorrência de estoque.
- create_all em vez de migrations (Alembic).
- Promoções/campanhas documentadas como regra, não implementadas como endpoint.
- Sem refresh token / logout server-side; sem anonimização LGPD.

Apresentar limitações com a solução proposta demonstra maturidade técnica e costuma valer pontos em banca/entrevista.